

732G57 Maskininlärning för statistiker

Föreläsning 10

Josef Wilzén

IDA, Linköping University, Sweden

- Projekt
- K-medoid klustring
- Densitetsbaserade metoder
- Faktorer som påverkar klusteranalys
- Utvärdera klusteranalys

Projekt och resten av kursen

Kursvecka 7 och framåt:

- Arbeta med projektet
- Datorlabbar: hjälp med projektet → utnyttja tiden!
- Förbereda inför tentan

Projektet - [kurshemsidan](#)

Viktiga dokument att ha koll på:

- Generella instruktioner: [länk](#)
- Datahatering och modellering: [länk](#)
- Mallen, finns info i: [pdf](#) och [rmd](#)
- Datum och deadlines: [länk](#)

K-medoid klustering - PAM

Använder **medoider** som center/prototyp vid klustering.

- En **medoid** är en representativ observation inom ett dataset/kluster.
- Medoid är **inte** samma som centroid, median, geometrisk median etc.
- Medoider är **lätta att tolka**
 - centroider kan vara punkter som inte liknar någon av observationerna i data.
- k-medoids:
 - minimerar summan av parvisa avstånd.
 - kan använda godtyckligt avståndsmått.
 - mer robust med brus och extremvärden.
- k-means: använder oftast euklidiskt avstånd.
- k-medoid klustering kallas också Partitioning Around Medoids (PAM)

PAM-algoritmen: BUILD och SWAP

PAM söker K medoider som minimerar summan av avstånden till observationernas närmaste medoid.

1. **BUILD:** Välj stegvis K observationer som initiala medoider.
2. Skapa kluster genom att tilldela varje observation till sin närmaste medoid.
3. **SWAP:** Undersök om en medoid kan bytas mot en observation som inte är medoid.
4. Genomför det byte som minskar den totala kostnaden mest.
5. Upprepa tills inget byte längre förbättrar lösningen.

Medoiderna är alltid faktiska observationer i datamängden.

Målet är att välja K bra startmedoider.

1. Välj först den observation som ger lägst total kostnad om den används som enda medoid.
 2. Välj därefter den observation som ger den största minskningen av den totala kostnaden. Spara som nästa medoid.
 3. Upprepa tills K medoider har valts.
- BUILD är en girig (greedy) algoritm.
 - Försöker sprida ut medoiderna över datamängden.
 - Ger ofta betydligt bättre startvärden än ett rent slumpmässigt val.

Efter BUILD-fasen försöker PAM förbättra lösningen.

1. Välj en aktuell medoid.
2. Välj en observation som inte är medoid.
3. Beräkna hur den totala kostnaden påverkas om de två observationerna byter plats.
4. Genomför det byte som ger störst förbättring.
5. Upprepa tills inget byte längre minskar kostnaden.

SWAP-fasen spelar en liknande roll som centroiduppdateringen i K-means.

PAM-algoritmen - detaljer

Givet

$$X = \{x_1, \dots, x_n\}, \quad d(\cdot, \cdot), \quad K,$$

söker PAM efter K medoider: $c_1, \dots, c_K$ som minimerar

$$L(c_1, \dots, c_K) = \sum_{i=1}^n \min_{k=1, \dots, K} d(x_i, c_k).$$

1. $c_1, \dots, c_K \leftarrow \text{BUILD}(X)$
2. Tilldela varje observation till närmaste medoid.
3. while(en förbättrande swap existerar)
 - $\text{SWAP}(c_1, \dots, c_K)$
 - Tilldela observationerna till närmaste medoid.
4. returnera $c_1, \dots, c_K$

PAM: SWAP-fasen - detaljer

Låt

$$M = \{c_1, \dots, c_K\}$$

vara de aktuella medoiderna.

- Undersök alla möjliga byten

$$c \in M, \quad x \notin M.$$

- Antal kandidatswaps: $K(n - K)$.
- För varje swap beräknas den nya kostnaden L .
- Genomför den swap som minskar L mest.
- Upprepa tills ingen förbättring är möjlig.

Tidskomplexitet

PAM: ungefär $\mathcal{O}(Kn^2)$ K-means: $\mathcal{O}(Kn)$

PAM med slumpmässiga startvärden

BUILD-fasen kan ersättas med slumpmässiga startvärden:

1. Välj K observationer slumpmässigt som startmedoider.
 2. Tilldela varje observation till sin närmaste medoid.
 3. Använd SWAP-fasen för att förbättra lösningen.
 4. Upprepa med flera slumpmässiga startvärden.
 5. Behåll den lösning som ger lägst total kostnad.
- Liknar användningen av `nstart` i K-means.
 - I `cluster::pam()` används `medoids = "random"` och `nstart`.

CLARA (Clustering Large Applications)

CLARA är en skalbar variant av PAM för stora datamängder.

1. Dra ett slumpmässigt stickprov av observationer.
 2. Kör PAM på stickprovet.
 3. Använd de erhållna medoiderna för att klustra hela datamängden.
 4. Upprepa för flera stickprov.
 5. Behåll den lösning som ger lägst total kostnad.
- Snabbare än PAM för stora datamängder.
 - Kan använda samma avståndsmått som PAM.
 - Resultatet beror på vilka stickprov som dras.

PAM kan använda många olika avståndsmått eftersom algoritmen endast kräver ett avstånd $d(\cdot, \cdot)$. Exempel

- **Cosinusavstånd**
- **Jaccard-avstånd**
- **Gower-avstånd**: Kan hantera numeriska, kategoriska, ordinala och binära variabler samtidigt.

Cosinusavstånd mäter skillnaden i **riktning** mellan två vektorer.

$$d(x, y) = 1 - \frac{x^\top y}{\|x\| \|y\|}$$

- Bygger på vinkeln mellan två vektorer.
- Oberoende av vektorernas längd.
- Två vektorer med samma riktning får avstånd 0.
- Två ortogonala vektorer får avstånd 1 (= mest olika)

Vanliga tillämpningar:

- Textdata
- Dokumentklustring.
- Embeddings från språk- och bildmodeller.

Jaccard-avstånd

Jaccard-avstånd används ofta för asymmetriska binära data.

$$d(x, y) = 1 - \frac{a}{a + b + c}$$

där

- a = antal variabler där båda observationerna har värdet 1,
- b = antal variabler där den första observationen har värdet 1 och den andra 0,
- c = antal variabler där den första observationen har värdet 0 och den andra 1.

Egenskaper:

- Gemensamma nollor påverkar inte avståndet. Fokuserar på närvaro snarare än frånvaro.

Vanliga tillämpningar: Marknadskorgsanalys, dokument representerade med ordindikatorer. Frånvaro/närvaro-data

Gower-avstånd

Gower-avstånd kan hantera blandade datatyper.

$$d_G(x_i, x_j) = \frac{\sum_{k=1}^p w_k d_{ijk}}{\sum_{k=1}^p w_k}$$

där d_{ijk} är avståndet för variabel k .

Egenskaper:

- Numeriska variabler skalas till intervallet $[0, 1]$.
- Kategoriska variabler behandlas via match/mismatch.
- Kan hantera ordinala och binära variabler.
- Alla delavstånd kombineras till ett totalavstånd.

Vanliga tillämpningar:

- Kundsegmentering.
- Enkätdata.
- Data med både numeriska och kategoriska variabler.

Densitetsbaserade metoder

Kluster kan formas baserat på hur densiteten på punkter varierar över variablerna: Täta områden kan defineras som ett kluster.



- Algoritm för att skapa kluster baserat på punkternas täthet.
- Använder två begrepp:
 - ϵ , en sökradie där vi letar efter punkter.
 - minPts, minsta antal punkter/grannar.

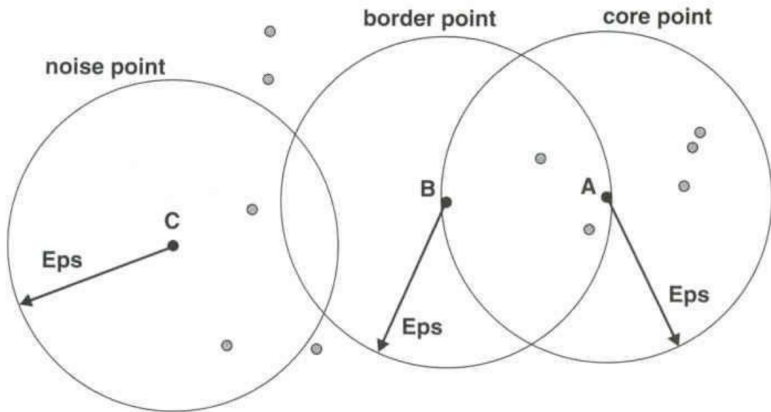
Från detta kan vi klassificera observationerna till någon av följande kategorier:

Kärnpunkt Punkter med minst minPts punkter inom sökradien ϵ .

Gränspunkt Inte en kärnpunkt men hamnar inom sökradien från en kärnpunkt.

Bruspunkt Varken kärnpunkt eller gränspunkt.

Illustration



Algorithm 8.4 DBSCAN algorithm.

- 1: Label all points as core, border, or noise points.
 - 2: Eliminate noise points.
 - 3: Put an edge between all core points that are within Eps of each other.
 - 4: Make each group of connected core points into a separate cluster.
 - 5: Assign each border point to one of the clusters of its associated core points.
-

Hyperparametrar som vi måste välja.

1. Definera ett nummer k .
2. Beräkna avståndet mellan varje punkt och dess k -närmaste granne och sortera punkterna enligt ökande avstånd.
3. Definera eps som värdet där skarp förändring märks (armbågsmetoden).
4. $\text{minPts} = k$.

k -värdet vi valt i steg 1 påverkar **inte** eps-värdet mycket om k inte är extremt (för litet eller för stort).

DBSCAN - Exempel



Figure 8.22. Sample data.

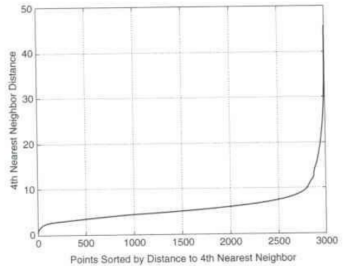
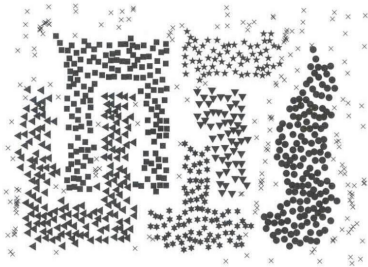
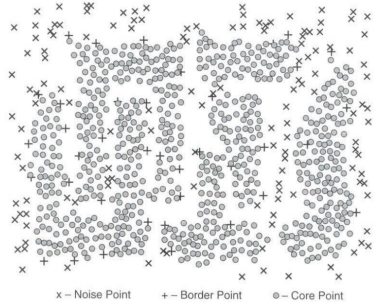


Figure 8.23. K-dist plot for sample data.

DBSCAN - Exempel



(a) Clusters found by DBSCAN.



(b) Core, border, and noise points.

- Brusbeständig.
- Behandlar kluster av olika former och storlekar.
- Problem med kluster som har tydligt varierande tätheter.
 - Svårt att välja ett bra eps.
- Problem i höga dimensioner.

Förbättring: [HDBscan](#)

- Variant som är en kombination av täthetsbaserad och hierarkisk klustring.
- Hyperparameter: Antal grannar, (eps bestäms automatiskt)
- Skalbar till stora dataset

K-means och DBSCAN

Egenskap	K-means	DBSCAN
Typ A	Partitionell	Partitionell
Typ C	Fullständig	Ofullständig
Klustertyp	Prototyp	Densitet
Klusterform	Klot	Olika
Närhetsmått	Olika	Olika
Användande av attribut	Alla	Alla
Upprepade körningar	Kluster beror på start-centroider	Samma kluster bildas
Algoritmbehov	k för antal kluster	eps och $minPts$
Optimeringsmodell	Ja	Nej
Tidskomplexitet	$O(m)$	$O(m^2)$

Faktorer som påverkar klusteranalys

- Dimensionalitet (problem för täthetsbaserade metoder).
- Datamängdens storlek (stora datamängder är svåra att skala upp).
- Brus och extremvärden.
- Skalan på data: numerisk, kategorisk.
 - problem att välja närhetsmått för datamängder med blandade attribut.
- Standardisering av variabler.

- Fördelningar - Olika metoder passar bättre på vissa fördelningar.
- Form - Godtyckliga former är svårare att klustra.
- Storlek - K-means, problem med olika storlekar.
- Täthet - Olika täthet problem för K-means, DBSCAN.
- Dåligt separerade kluster - Vissa metoder slår ihop överlappande kluster eller kluster som ligger nära varandra

Ingen klustermetod passar för alla dataset!

Utvärdera klusteranalys

- Finns det kluster i data? Eller har observationerna bara slumpmässiga värden?
 - Notera: det kan finnas bra ett "naturligt" kluster i data
- Avgöra rätt antal kluster.
- Interna mått på hur bra klusteranalysen är.
- Externa mått på hur bra klusteranalysen är → om vi har tillgång till sanna klasser/grupper.
- Jämföra olika metoder för klusteranalys på samma dataset.
- Kontext och problembeskrivning: avgör om vi har en bra klustring. Hjälper resultatet från klustringen oss att lösa problemet?

Interna mått.

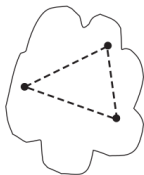
Cohesion: Hur tigt eller sammanhållet ett kluster är med sig själv.

Separation: Hur väl separerat ett kluster är från övriga kluster.

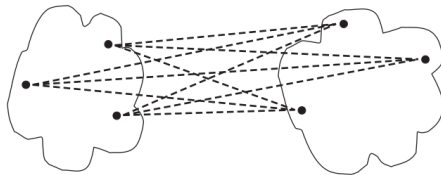
När vi har beräknat dessa mått för ett kluster kan vi väga samman alla dessa mått till ett mått för hela klustringen.

Cohesion och Separation

$$\text{cohesion}(C_i) = \sum_{x \in C_i, y \in C_i} \text{proximity}(x, y)$$
$$\text{proximity}(C_i, C_j) = \sum_{x \in C_i, y \in C_j} \text{proximity}(x, y)$$



(a) Cohesion.



(b) Separation.

Figure 7.27. Graph-based view of cluster cohesion and separation.

$\text{proximity}(x, y)$ kan vara både närhetsmått eller avståndsmått.

The Silhouette Coefficient

Använder både cohesion och separation för att beräkna ett mått.

1. Beräkna medelavståndet från observation_{*i*} till alla andra observationer i dess kluster, kalla det a_i .
2. Beräkna nu medelavståndet från observation_{*i*} till alla kluster som inte innehåller denna observation.
3. Hitta det minsta av dessa avstånd kalla det b_i .
4. Silhouette coefficient för observation_{*i*} defineras som,

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

The Silhouette Coefficient

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

- s_i kan ta värden mellan -1 och 1 .
- 1 är bästa möjliga värde.
 - Vill ha $a_i < b_i$ och att a_i ska vara nära noll.
- Average silhouette coefficient.
 - Ta medelvärdet över alla s_i
 - Ger ett mått på hur bra klustringen är.

The Silhouette Coefficient - Exempel

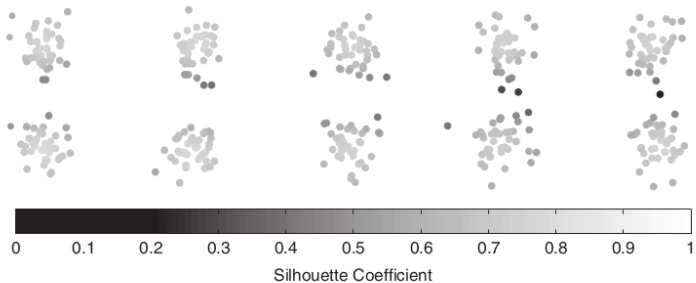


Figure 7.29. Silhouette coefficients for points in ten clusters.

- K-means: vi kan använda total SSE och average silhouette coefficient.
- Plotta dessa mot antal kluster.
 - Kolla efter böjar och toppar.
 - SSE planar ut efter en böj: ta antal kluster vid böjen.
 - Average silhouette coefficient: Kolla om det finns en eller flera toppar.

Välja antal kluster - Exempel

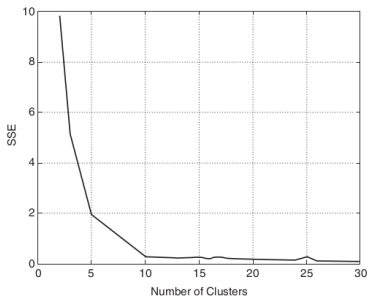


Figure 7.32. SSE versus number of clusters for the data of Figure 7.29 on page 582.

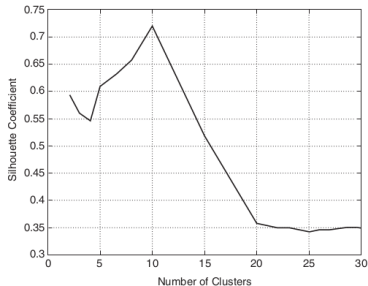


Figure 7.33. Average silhouette coefficient versus number of clusters for the data of Figure 7.29.

Calinski-Harabasz Index

Inter-cluster dispersion

$$\text{BCSS} = \sum_{k=1}^K n_k \|C_k - C\|^2.$$

Intra-cluster dispersion

$$\text{WCSS}_k = \sum_{i=1}^{n_k} \|x_{i,k} - C_k\|^2, \quad \text{WCSS} = \sum_{k=1}^K \text{WCSS}_k.$$

Calinski-Harabasz Index

$$\text{CH} = \frac{\text{BCSS}}{\text{WCSS}} \cdot \frac{N - K}{K - 1}.$$

Höga värden är bra för CH.

Davies-Bouldin Index är ett liknande mått. Där ska man ha låga värden.

Välja antal kluster

- Vi kan beräkna närhetsmatrisen eller avståndsmatrisen för alla datapunkter.
 - Matris med alla parvisa närheter/avstånd mellan observationer.
- Notera att detta är dyrt!
 - Kostar $O(n^2)$
 - Svårt att plotta med många observationer.
 - En lösning är att ta ett slumpmässigt urval av data.
- Sortera närhetsmatrisen baserat på kluster.
 - Först kommer kluster 1, sen kluster 2, osv.
- Om vi har väl separerade kluster och valt ett bra antal kluster kommer den sorterade matrisen vara ungefär blockdiagonal.

Välja antal kluster - Exempel

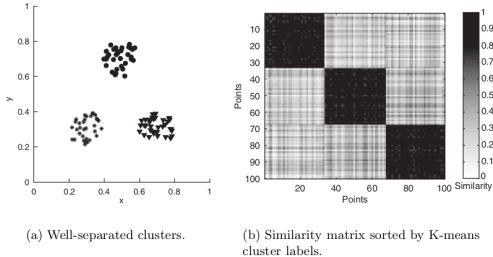


Figure 7.30. Similarity matrix for well-separated clusters.

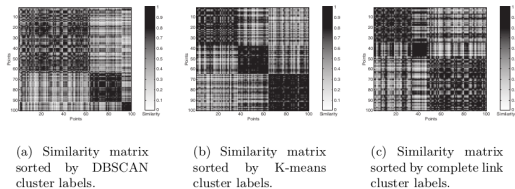


Figure 7.31. Similarity matrices for clusters from random data.

- Jämför med sanna klasser/kluster.
- Vi kan ta resultatet från vår klusteranalys som våra "predikterade värden"
- Kan då jämföra med sanna klasserna.
 - Vi kan då beräkna förväxlingsmatris och liknande mått.
- Notera:
 - Vi har inte de "rätta namnen" på våra kluster.
 - Vi vill ofta att klustren ska vara så rena som möjligt, dvs. domineras av en klass.

Användning and utvärdering av klusteranalys

- Börja med lämplig datahantering: välja variabler, skalning, mm
- Välj lämpligt avståndsmått/närhetsmått
- Överväg om någon variabelreduktion kan behövas (väja mindre delmängd av variabler, PCA, autoencoders, etc)
- Ofta är det bra att testa olika klusteringsmetoder och olika inställningar på hyperparametrar
- Vi kan använda klustering för att diskretisera variabler
 - Välj en eller ett mindre antal variabler
 - Kör en klustring
 - Använd klustertillhörighet som en ny kategorisk variabel

Användning and utvärdering av klusteranalys

- Klusteranalys kan användas för att komprimera ett dataset. Här passar det bra med någon prototypbaserad klustering
 - Om vi har många obs så kör vi k-means eller PAM på data, där k är relativt stort
 - Vi använder sen centroiderna/medoiderna som data i vidare analyser: vi har nu k stycken obs
 - Vi kan använda den "nya datan" för klustering med någon annan klustringsmetod eller för annan dataanalys
- För att testa robusthet i resultatet:
 - Dra ett antal slumpmässiga urval från data (t.ex. med 70-90% av obs i ursprungliga data)
 - Klustra alla dessa data med samma metod med samma inställningar
 - Undersök om klustering blir lika mellan dataseten eller om det är tydliga skillnader